Concurrent Execution

Concurrent execution means running parts of multiple transactions at the same time in an interleaved manner.
Instead of one transaction completing fully before another starts, the DBMS mixes the operations of different transactions.

This approach:

- Improves CPU and resource utilization
- Increases system throughput
- Reduces waiting time for transactions

Multiple transactions can make progress simultaneously without waiting for others to finish.

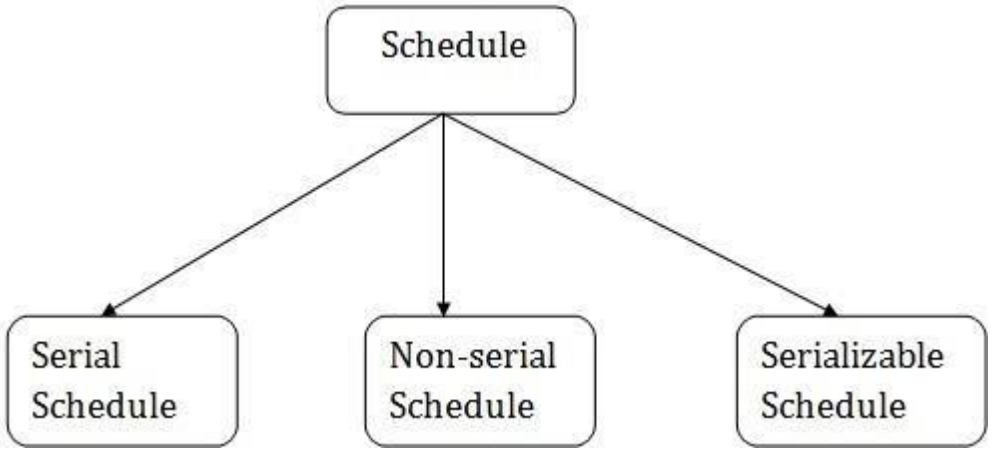
Example:
While one transaction is reading data from disk, another transaction may be writing or performing computation.

Benefits

- Helps in reducing waiting time
- Improved throughput and resource utilization

SCHEDULE

A series of operations from one transaction to another transaction is known as schedule. It is used to preserve the order of the operation in each of the individual transactions.



1. **Serial Schedules:**

   Schedules in which the transactions are executed non-interleaved, i.e., a serial schedule is one in which no transaction starts until a running transaction has ended are called serial schedules.

   Example: Consider the following schedule involving two transactions T_1 and T_2.

T_1	T_2
R(A)	
W(A)	
R(B)	
	W(B)
	R(A)
	R(B)

Where R(A) denotes that a read operation is performed on some data item 'A'.
This is a serial schedule since the transactions perform serially in the order T_1 —> T_2.

2. Non-serial Schedule

In a non-serial Schedule, multiple transactions execute concurrently/simultaneously, unlike the serial Schedule, where one transaction must wait for another to complete all its operations. In the Non-Serial Schedule, the other transaction proceeds without the completion of the previous transaction. All the transaction operations are interleaved or mixed with each other.

Transaction T1	Transaction T2
R1(A)	
W1(A)	
	R2(A)
	W2(A)
R1(B)	
W1(B)	
	R2(B)
	W2(B)

S1 (Schedule 1)

In this Schedule, there are two transactions, T1 and T2, executing concurrently. The operations of T1 and T2 are interleaved. So, this Schedule is an example of a Non-Serial Schedule.

3. Serializability

- When many transactions run concurrently, the database may become inconsistent if not properly controlled.
- Serializability is a concept used to check whether a concurrent schedule is correct and safe.
- A schedule is serializable if its result is equivalent to the result of executing the transactions one by one (serially).
- It ensures that the database remains consistent even when transactions execute concurrently.

There are two types of serializability:

1. Conflict serializability
2. View serializability

Conflict Serializability

It is one of the types of Serializability, which can be used to check whether a non-serialschedule is conflict serializable or not.

If any non-serial schedule is conflict equivalent to serial schedule, then it is called as conflict serializable schedule.

S_1

T_1	T_2
R(A)	
W(A)	
	R(A)
	W(A)
R(B)	
W(B)	
	R(B)
	W(B)

S_2

T_1	T_2
R(A)	
W(A)	
R(B)	
W(B)	
	R(A)
	W(A)
	R(B)
	W(B)

For S_1

$R_1(A)$ is followed by $W_2(A)$

$W_1(A)$ is followed by $R_2(A)$

$W_1(A)$ is followed by $W_2(A)$

$R_1(B)$ is followed by $W_2(B)$

$W_1(B)$ is followed by $R_2(B)$

$W_1(B)$ is followed by $W_2(B)$

Hence **S_1** is **conflict serializable schedule**

For S_2

$R_1(A)$ is followed by $W_2(A)$

$W_1(A)$ is followed by $R_2(A)$

$W_1(A)$ is followed by $W_2(A)$

$R_1(B)$ is followed by $W_2(B)$

$W_1(B)$ is followed by $R_2(B)$

$W_1(B)$ is followed by $W_2(B)$

Test for conflict serializability:

To check conflict serializability, precedence graph is used.

Let 'S' be a schedule, construct a directed graph known as precedence graph. Graph consists of a pair of G = (V, E)

Where,

V : a set of vertices

E : a set of edges

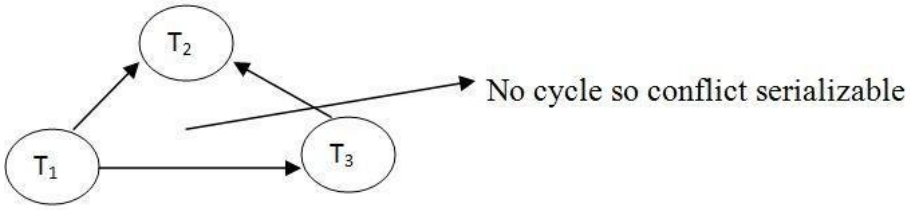
Algorithm for creating a precedence graph

Step 1: Create a node for each transaction

Step 2: Draw a directed edge from T_i to T_j , if T_j reads a value of an item written by T_i.

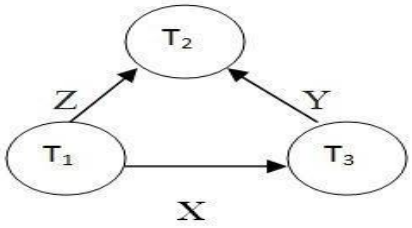
Step 3: Draw a directed edge from T_i to T_j , if T_j writes a value into an item after it has been read by T_i.

Step 4: Draw a directed edge from T_i to T_j , if T_j writes after T_i write.



1 : Check for conflict serializability

T_1	T_2	T_3
R(X)		
	R(Z)	
R(Z)		
		R(X)
		R(Y)
		W(X)
	R(Y)	
	W(Z)	
	W(Y)	



No Cycle
Conflict Serializable

T_1	T_2	T_3
R(X)		
		R(X)
		W(X)
W(X)		
	R(X)	

Example 2 : Check for conflict serializability

2.View Serializability:

View Serializability is a process to find out that a given schedule is view serializable or not.

To check whether a given schedule is view serializable, we need to check whether the given schedule is View Equivalent to its serial schedule. Let's take an example to understand what I mean by that.

Given Schedule:

```
T1       T2
-----    ------
R(X)
W(X)
         R(X)
         W(X)
R(Y)
W(Y)
         R(Y)
         W(Y)
```

Serial Schedule of the above given schedule:
As we know that in Serial schedule a transaction only starts when the current running transaction is finished. So the serial schedule of the above given schedule would look like this:

```
T1       T2
-----    ------
R(X)
W(X)
R(Y)
W(Y)
         R(X)
         W(X)
         R(Y)
         W(Y)
```

If we can prove that the given schedule is **View Equivalent** to its serial schedule then the given schedule is called **view Serializable**.

Why do we need View Serializability?
A serial schedule means transactions run one after another, not at the same time. This is always safe and keeps the database correct, but it's slow.
A non-serial schedule lets transactions run at the same time (concurrently). This makes

better use of system resources and is much faster. But if not handled properly, it can lead to wrong results and make the database inconsistent.
To avoid that, we use view serializability to check whether a non-serial schedule is still safe that is, it gives the same result as some serial schedule. If it is view serializable, we know the database will stay correct even with concurrent transactions.
So, even though serial schedules are always safe, we don't always use them because they're slow. Instead, we allow faster, concurrent execution and use view serializability to make sure it's still safe.

View Equivalent:
Lets learn how to check whether the two schedules are view equivalent. Two schedules T1 and T2 are said to be view equivalent, if they satisfy all the following conditions:

1. Initial Read
An initial read of both schedules must be the same

T1	T2
Read(A)	
	Write(A)

Schedule S1

T1	T2
	Write(A)
Read(A)	

Schedule S2

Suppose two schedule S1 and S2. In schedule S1, if a transaction T1 is reading the data item A, then in S2, transaction T1 should also read A. Above two schedules are view equivalent because Initial read operation in S1 is done by T1 and in S2 it is also done by T1.

2. Updated Read
In schedule S1, if Ti is reading A which is updated by Tj then in S2 also, Ti should read A which is updated by Tj.

T1	T2	T3
Write(A)		
	Write(A)	
		Read(A)

Schedule S1

T1	T2	T3
	Write(A)	
Write(A)		
		Read(A)

Schedule S2

Above two schedules are not view equal because, in S1, T3 is reading A updated by T2 and in S2, T3 is reading A updated by T1.

3. Final Write

A final write must be the same between both the schedules. In schedule S1, if a transaction T1 updates A at last then in S2, final writes operations should also be done by T1.

T1	T2	T3
Write(A)		
	Read(A)	
		Write(A)

Schedule S1

T1	T2	T3
	Read(A)	
Write(A)		
		Write(A)

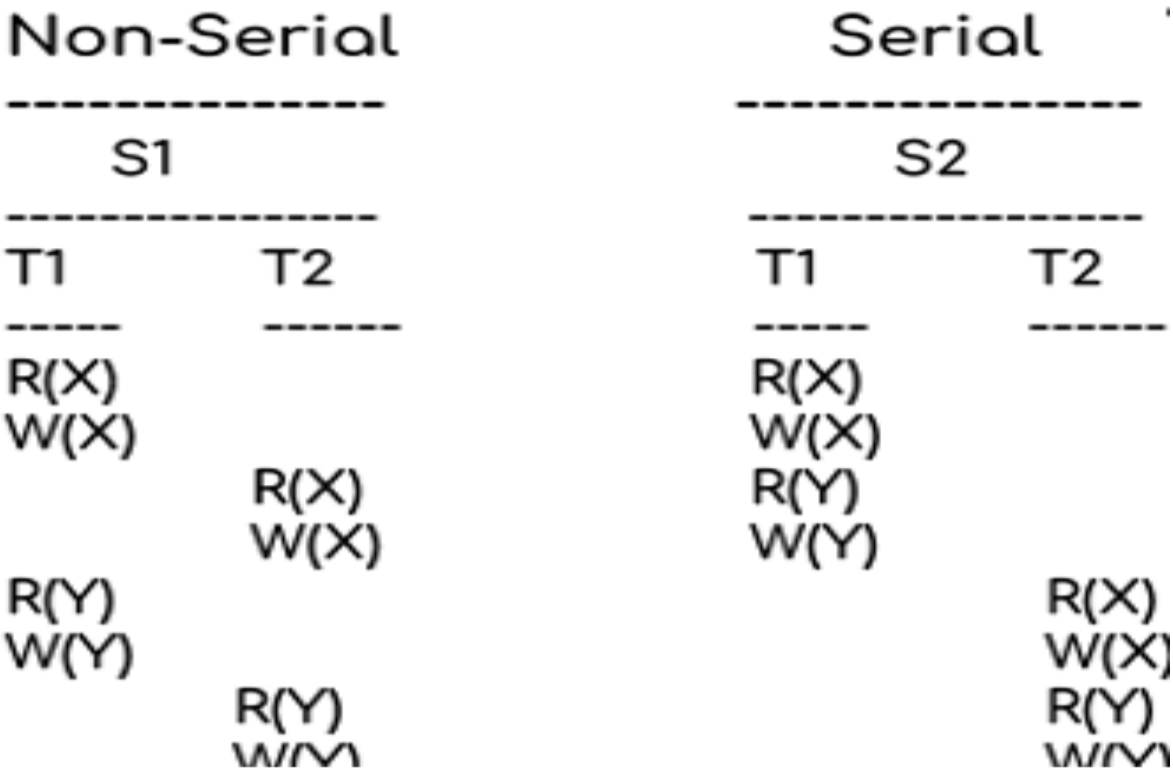
Schedule S2

Above two schedules is view equal because Final write operation in S1 is done by T3 and in S2, the final write operation is also done by T3.

View Serializable

If a schedule is view equivalent to its serial schedule then the given schedule is said to be View Serializable. Lets take an example.

View Serializable Example



S2 is the serial schedule of S1. If we can prove that they are view equivalent then we can say that given schedule S1 is view Serializable. Let's check the three conditions of view serializability:

1. **Initial Read**

In schedule S1, transaction T1 first reads the data item X. In S2 also transaction T1 first reads the data item X.

Let's check for Y. In schedule S1, transaction T1 first reads the data item Y. In S2 also the first read operation on Y is performed by T1. We checked for both data items X & Y and the initial read condition is satisfied in S1 & S2.

2. **Final Write**

In schedule S1, the final write operation on X is done by transaction T2. In S2 also transaction T2 performs the final write on X.

Lets check for Y. In schedule S1, the final write operation on Y is done by transaction T2. In schedule S2, final write on Y is done by T2.

We checked for both data items X & Y and the final write condition is satisfied in S1 & S2.

3. **Update Read**

In S1, transaction T2 reads the value of X, written by T1. In S2, the same transaction T2 reads the X after it is written by T1.

In S1, transaction T2 reads the value of Y, written by T1. In S2, the same transaction T2 reads the value of Y after it is updated by T1.

The update read condition is also satisfied for both the schedules.

Result: Since all the three conditions that checks whether the two schedules are view equivalent are satisfied in this example, which means S1 and S2 are view equivalent. Also, as we know that the schedule S2 is the serial schedule of S1, thus we can say that the schedule S1 is view serializable schedule.

Lock Based Protocol

A Lock-Based Protocol is a concurrency control method used in DBMS to manage simultaneous transactions and ensure data consistency. It prevents conflicts when two or more transactions try to access the same data item at the same time.

Types of Locks in DBMS

There are two types of Locking protocol:

- Shared Lock (S)
- Exclusive Lock (X)

Let's discuss these two types of locks in detail:

Shared Lock (Read Lock):

A shared lock allows transactions to read the same data at the same time, but no transaction can modify it while the lock is held.

Example:

If two transactions want to read a person’s account balance, both can do so using a shared lock. However, no other transaction can update the account balance until both reading transactions finish.

Exclusive Lock (X-Lock):
An exclusive lock allows a transaction to read and write a data item. While this lock is active, no other transaction can read or write the same data.

Example:
When a transaction updates a person's account balance, it places an exclusive lock on that account. This prevents any other transaction from accessing the account until the update is completed.

Shared Lock	Exclusive Lock
• Used when a transaction only reads a data item. • **Allowed:** Multiple transactions can hold shared locks at the same time. • **No conflict** with other shared locks. • **Compatible** with other shared locks.	• Used when a transaction wants to read and **write (update)** a data item. • **Not allowed:** Only one transaction can hold an exclusive lock. • **Conflicts** with both shared and exclusive locks. • **Incompatible** with any other lock (S or X).

Lock Compatibility Matrix
A Lock Compatibility Matrix is a table used in database management systems (DBMS) to determine if a transaction's request for a lock on a data item can be granted when other transactions already hold locks on the same item. It defines which types of locks are compatible with each other and which ones conflict.

	S	X
S	✔	✗
X	✗	✗

There are four types of lock protocols available:

1. Simplistic lock protocols

Simplistic (or basic) lock protocols are a simple way to control concurrent access to shared data in a DBMS. The main idea is that a transaction must lock a data item before accessing it and release the lock after use**,** ensuring that conflicts between transactions are avoided. The core principle of this protocol is straightforward:

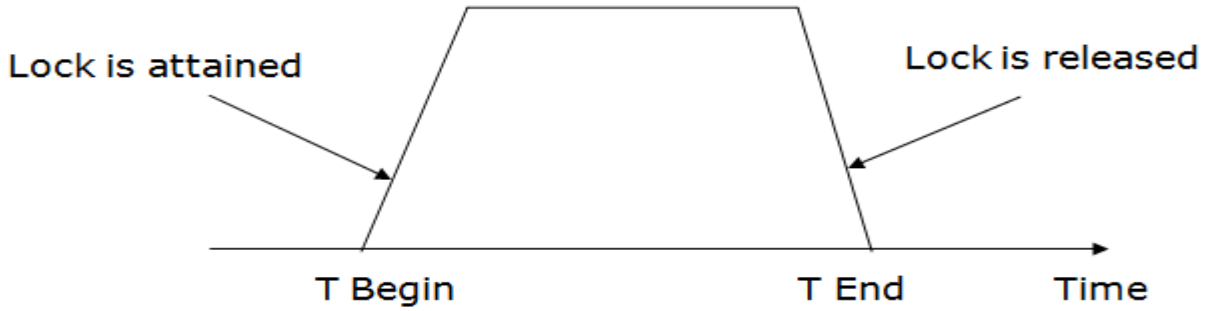
- **Acquire a lock before access:** Every transaction must obtain a lock on a data item before reading or writing it.
- **Release lock after operation:** Once the transaction completes its read or write operation on the data item, it releases the lock

2. Pre-claiming Lock Protocol

- In Pre-claiming Lock Protocol, a transaction must declare in advance all the data items it will need to access.
- The system tries to lock all of them before the transaction starts.
- If all locks can be granted, the transaction proceeds.
- If even one lock is unavailable, the transaction waits and does not start at all.

3. Two-phase locking (2PL)

- The two-phase locking protocol divides the execution phase of the transaction into three parts.
- In the first part, when the execution of the transaction starts, it seeks permission for the lock it requires.
- In the second part, the transaction acquires all the locks. The third phase is started as soon as the transaction releases its first lock.
- In the third phase, the transaction cannot demand any new locks. It only releases the acquired locks.



There are two phases of 2PL:

Growing phase: In the growing phase, a new lock on the data item may be acquired by the transaction, but none can be released.

Shrinking phase: In the shrinking phase, existing lock held by the transaction may be released, but no new locks can be acquired.

In the below example, if lock conversion is allowed then the following phase can happen:

1. Upgrading of lock (from S(a) to X (a)) is allowed in growing phase.
2. Downgrading of lock (from X(a) to S(a)) must be done in shrinking phase.

Example:

	T1	T2
0	LOCK-S(A)	
1		LOCK-S(A)
2	LOCK-X(B)	
3	——	——
4	UNLOCK(A)	
5		LOCK-X(C)
6	UNLOCK(B)	
7		UNLOCK(A)
8		UNLOCK(C)
9	——	——

The following way shows how unlocking and locking work with 2-PL.

Transaction T1:

- **Growing phase:** from step 1-3
- **Shrinking phase:** from step 5-7
- **Lock point:** at 3

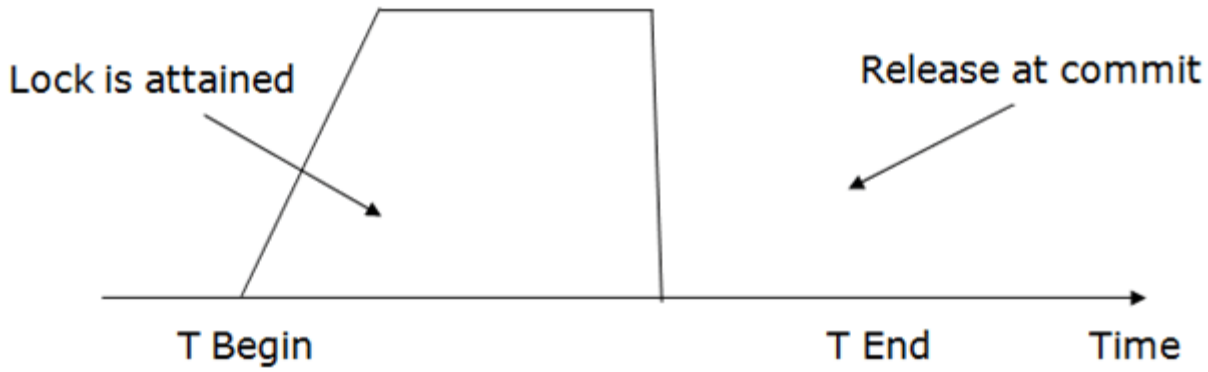
Transaction T2:

- **Growing phase:** from step 2-6
- **Shrinking phase:** from step 8-9
- **Lock point:** at 6

4. Strict Two-phase locking (Strict-2PL)

- The first phase of Strict-2PL is similar to 2PL. In the first phase, after acquiring all the locks, the transaction continues to execute normally.

- The only difference between 2PL and strict 2PL is that Strict-2PL does not release a lock after using it.
- Strict-2PL waits until the whole transaction to commit, and then it releases all the locks at a time.
- Strict-2PL protocol does not have shrinking phase of lock release.



Timestamp Ordering Protocol:

- The Timestamp Ordering Protocol is used to order the transactions based on their Timestamps. The order of transaction is nothing but the ascending order of the transaction creation.
- The priority of the older transaction is higher that's why it executes first. To determine the timestamp of the transaction, this protocol uses system time or logical counter.
- The lock-based protocol is used to manage the order between conflicting pairs among transactions at the execution time. But Timestamp based protocols start working as soon as a transaction is created.
- Let's assume there are two transactions T1 and T2. Suppose the transaction T1 has entered the system at 007 times and transaction T2 has entered the system at 009 times. T1 has the higher priority, so it executes first as it is entered the system first.
- The timestamp ordering protocol also maintains the timestamp of last 'read' and 'write' operation on a data.

Basic Timestamp ordering protocol works as follows:

1. Check the following condition whenever a transaction Ti issues a **Read (X)** operation:

- If W_TS(X) >TS(Ti) then rollback Ti.
- Otherwise execute R(A) operation.
- Set RTS(A)= max{ RTS(A),Ts(Ti)}

2. Check the following condition whenever a transaction Ti issues a **Write(X)** operation:

- If TS(Ti) < R_TS(X) then rollback Ti.
- If TS(Ti) < W_TS(X) then rollback Ti.
- Otherwise execute write(A) operation.

  Set WTS(A)= Ts(Ti)

Where,

TS(Ti) denotes the timestamp of the transaction Ti.

R_TS(X) denotes the Read time-stamp of data-item X.

W_TS(X) denotes the Write time-stamp of data-item X.

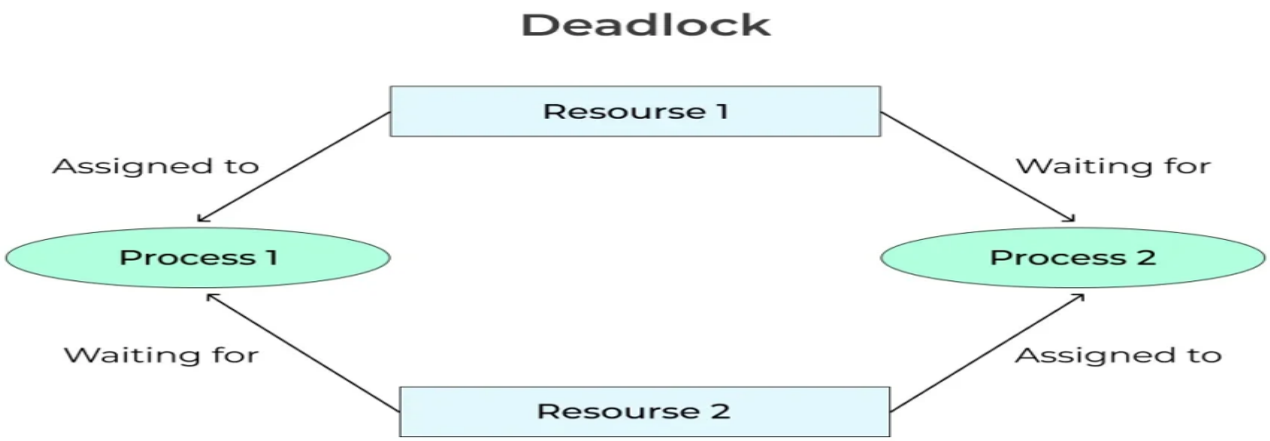
Example:

T_1	T_2	T_3
R(A)		
	R(B)	
W(C)		
		R(B)
R(C)		
	W(B)	
		W(A)

	A	B	C
RTS	0	0	0
WTS	0	0	0

Deadlock in DBMS:

In a database, a deadlock is a situation in which two or more transactions are waiting for one another to give up locks.



Example:
Transaction T1 is waiting for transaction T2 to release the lock and similarly transaction T2 was waiting for transaction T1 to release It's lock, as a result, all activities come to halt state. This continues until the DBMS detects that deadlock has occurred and abort some of the transactions.

Necessary conditions for Deadlocks

- **Mutual Exclusion**: A resources can only be used by one process at a time. Think of a one-lane bridge that only one car can use at once.

- **Hold and Wait**: A process can hold a number of resources at a time and at the same time, it can request for other resources that are being held by some other process. It's like a car on the bridge waiting for another car to move off so it can continue.

- **No pre-emption**

A resource cannot be taken back forcefully from a transaction; it is released only when the transaction finishes**.**

- **Circular Wait**

All the processes must be waiting for the resources in a cyclic manner so that the last process is waiting for the resource which is being held by the first process.

Deadlock Avoidance:
Deadlock avoidance mechanism is used to detect any deadlock situation in advance by using some techniques like WEIGHTS FOR GRAPH, but this wait for graph method is suitable only for smaller databases, for large databases other prevention techniques are used

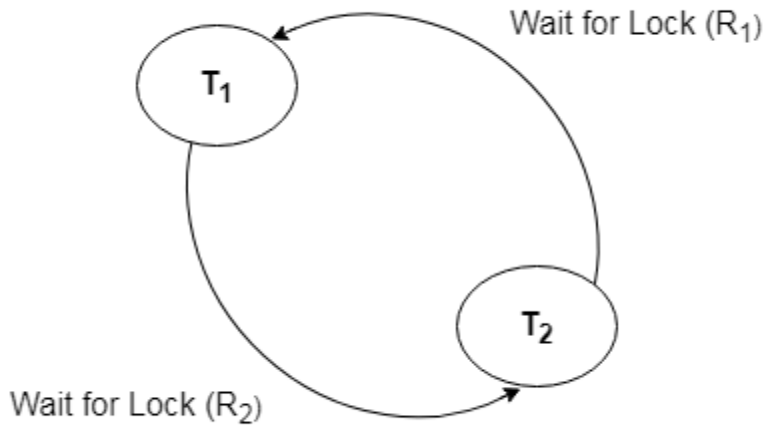
Deadlock Detection:

In a database, when a transaction waits indefinitely to obtain a lock, then the DBMS should detect whether the transaction is involved in a deadlock or not. The lock manager maintains a Wait for the graph to detect the deadlock cycle in the database.

Wait for Graph:

- This is the suitable method for deadlock detection. In this method, a graph is created based on the transaction and their lock. If the created graph has a cycle or closed loop, then there is a deadlock.
- The wait for the graph is maintained by the system for every transaction which is waiting for some data held by the others. The system keeps checking the graph if there is any cycle in the graph.

The wait for a graph for the above scenario is shown below:



Deadlock Prevention

- Deadlock prevention method is suitable for a large database. If the resources are allocated in such a way that deadlock never occurs, then the deadlock can be prevented.
- The Database management system analyzes the operations of the transaction whether they can create a deadlock situation or not. If they do, then the DBMS never allowed that transaction to be executed.

Wait-Die scheme

The Wait–Die scheme is a method used by DBMS to prevent deadlocks using timestamps.

- Every transaction is given a timestamp when it starts.
- Older transaction = smaller timestamp
- Younger transaction = larger timestamp

How it works:

When a transaction requests a resource that is already locked by another transaction, the DBMS compares their timestamps.

- If the requesting transaction is older: it is allowed to wait
- If the requesting transaction is younger: it is aborted (dies) and restarted later with the same timestamp

Let's assume there are two transactions Ti and Tj and let TS(T) is a timestamp of any transaction T. If T2 holds a lock by some other transaction and T1 is requesting for resources held by T2 then the following actions are performed by DBMS:

1. Check if TS(Ti) < TS(Tj) - If Ti is the older transaction and Tj has held some resource, then Ti is allowed to wait until the data-item is available for execution. That means if the older transaction is waiting for a resource which is locked by the younger transaction, then the older transaction is allowed to wait for resource until it is available.
2. Check if TS(T_i) < TS(Tj) - If Ti is older transaction and has held some resource and if Tj is waiting for it, then Tj is killed and restarted later with the random delay but with the same timestamp.

Wound wait scheme

- In wound wait scheme, if the older transaction requests for a resource which is held by the younger transaction, then older transaction forces younger one to kill the transaction and release the resource. After the minute delay, the younger transaction is restarted but with the same timestamp.
- If the older transaction has held a resource which is requested by the Younger transaction, then the younger transaction is asked to wait until older releases it.

ACID properties of transaction:

The ACID properties are a set of fundamental characteristics that guarantee database transactions are processed reliably.

1. Atomicity

- A transaction is treated as a single unit, which either completes entirely or doesn't happen at all.
- **Example**: Transferring money from Account A to Account B if debit from A is done but credit to B fails, the whole transaction is rolled back.

2. Consistency

- A transaction must bring the database from one valid state to another valid state, maintaining all predefined rules, constraints, and integrity.
- The database must stay correct before and after a transaction.

Example: If a rule says "balance can't be negative," any transaction that breaks this rule is not allowed.

3. Isolation

- Transactions executing concurrently should not interfere with each other. Each transaction should appear as if it were executed independently.

- **Example**: If two people try to book the same train seat, only one should succeed each booking works as if it's the only one happening.

4. Durability

- Once a transaction is committed, its effects are permanent, even in the case of a system crash.
- **Example**: If you transfer money and get confirmation, the change in balance will persist even after a power failure.

1) What is the need for transaction processing in a database system?
2) Explain the significance of each ACID property in maintaining transaction integrity.
3) What is a schedule in transaction processing? How does serializability ensure the correctness of a schedule?
4) Explain the concept of serializability in transaction scheduling. Why is it important?
5) What is a locking mechanism in a DBMS? Explain how lock-based protocols help in concurrency control.
6) Discuss different types of locks used in locking-based concurrency control protocols.
7) Explain the concept of time-stamping-based concurrency control protocols. How does it ensure transaction consistency?
8) What is a deadlock in a database system? Explain two common methods of handling deadlocks.